Glava 2

C++ niti i podela posla

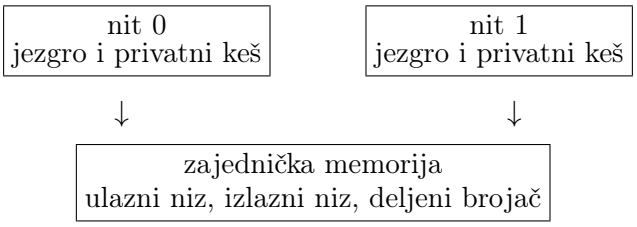
Cilj vežbe

- Kreirati niti, podeliti im posao i sačekati rezultate.
- Paralelizovati direktnu pokretnu srednju vrednost iz prve vežbe.
- Prepoznati trku i objasniti kada je potrebno zaključavanje.
- Izmeriti ubrzanje i povezati ga sa radom jezgara i deljenom memorijom.

Komande pokretati iz `root_dir`. Glavni zadatak je da isti proračun pokretne srednje vrednosti podele niti. Kratki primeri sa ispisom, sumom i brojačem služe da se prvo razumeju niti.

2.1 Šta dele niti

Niti istog procesa dele adresni prostor: mogu pristupati istom nizu. Svaka nit ima svoj stek, lokalne promenljive i stanje izvršavanja. Operativni sistem raspoređuje programske niti na logičke CPU-e. Jezgro sa SMT podrškom ima više hardverskih niti koje dele resurse jezgra. Zato četiri jezgra sa po dve hardverske niti daju osam logičkih CPU-a, ali ne i osam nezavisnih fizičkih jezgara. Ako program pokrene više niti nego što ima raspoloživih CPU-a, operativni sistem smenjuje njihovo izvršavanje.



Slika 2.1: Pojednostavljen prikaz dve niti na dva jezgra.

Keš čuva kopije delova memorije blizu jezgra. Podaci se prenose u grupama susednih bajtova, koje zovemo linije keša. Dva susedna elementa niza mogu biti u istoj liniji. Za sada je dovoljno razlikovati čitanje zajedničkog ulaza od čestog menjanja istog brojača; detalji keša slede u posebnoj vežbi.

2.2 Kreiranje i čekanje niti

2.2.1 Prvo sa običnom funkcijom

Naziv `worker` u kodu označava funkciju koju izvršava nova nit. U fajlu `src/threads/hello.cpp` ona samo ispisuje dobijeni broj:

```
void worker(int id)
{
    printf("Hello from worker %d\n", id);
}
```

Glavna nit priprema vektor i pokreće niti:

```
vector<thread> threads;
threads.reserve(num_workers);

for (int t = 0; t < num_workers; ++t)
{
    threads.emplace_back(worker, t);
}

printf("This is main thread\n");

for (auto& t : threads)
{
    t.join();
}
```

Poziv `emplace_back(worker, t)` stvara nit koja poziva `worker(t)`. Argument `t` se kopira, pa svaka nit dobija svoj ID čak i kada glavna nit nastavi petlju. Vrednost `t` je jedinstven ID u ovoj grupi niti, od 0 do `num_workers - 1`. Operativni sistem određuje na kojem logičkom CPU-u se nit izvršava. Poziv `reserve` unapred obezbeđuje mesta u vektoru.

Metoda `join` čeka završetak niti. Prvo pokrenuti sve niti, pa ih onda sve sačekati; čekanje u petlji za kreiranje sprečilo bi njihov zajednički rad. Redosled ispisa može se menjati između pokretanja.

```
g++ -O3 -Wall -Wextra -pthread \
    src/threads/hello.cpp -o src/thread_hello
./src/thread_hello 4
```

2.2.2 Zašto emplace_back

Metoda `emplace_back` pravi objekat niti neposredno u vektoru. Metoda `push_back` dobija već napravljen objekat. Sledeća dva zapisa su alternative:

```
threads.emplace_back(worker, t);

threads.push_back(thread(worker, t));
```

U drugoj varijanti privremeni objekat `thread` premešta se u vektor. Premeštanjem se prenosi objekat koji predstavlja već pokrenutu nit. Koristimo `emplace_back` jer je zapis kraći i direktno navodi funkciju i njene argumente.

2.2.3 Isti posao pomoću lambda izraza

Kratku funkciju možemo napisati na mestu gde je potrebna, bez posebnog imena. Takav zapis zove se lambda izraz. U fajlu `src/threads/hello_lambda.cpp` petlja za pokretanje izgleda ovako:

```
    for (int t = 0; t < num_workers; ++t)
    {
        threads.emplace_back([t]
        {
            printf("Hello from worker %d\n", t);
        });
    }
```

Deo `[t]` daje funkciji kopiju vrednosti `t` iz okolnog koda, a vitičaste zagrade sadrže njen posao. Nit ispisuje istu poruku kao funkcija `worker`; vektor i čekanje ostaju isti. Kompajlirati i pokrenuti lambda varijantu:

```
g++ -O3 -Wall -Wextra -pthread \
    src/threads/hello_lambda.cpp -o src/thread_hello_lambda
./src/thread_hello_lambda 4
```

Kada nit koristi i zajedničke objekte, zapis `[&, t]` znači: broj `t` kopiraj, ostale potrebne objekte koristi po referenci. Kopiranje indeksa čuva ID svake niti dok glavna nit nastavlja petlju. Deljeni objekti moraju postojati do završetka niti; veličinu njihovih nizova za to vreme ne menjati.

2.3 Kratak primer: podela sume

Program `src/threads/sum.cpp` sabira brojeve od 1 do N. Nit t dobija indekse od $\lfloor Nt/p \rfloor$ do $\lfloor N(t+1)/p \rfloor$, bez poslednjeg indeksa, gde je p broj niti. Na primer, za $N = 10$ i $p = 3$ intervali su $[0, 3)$, $[3, 6)$ i $[6, 10)$. Poslednja nit ima jednu iteraciju više; svih deset iteracija je dodeljeno. Za indekse koristimo `int`, a za sume uvodimo kraće ime tipa:

```
using Sum = long long;
```

Zapis `using` daje tipu `long long` ime `Sum`. Suma može biti mnogo veća od samog indeksa: za $N = 100000$ iznosi 5000050000, što prevazilazi opseg uobičajenog 32-bitnog `int`. Funkcija za jedan blok prima granice i mesto za rezultat:

```
void sum_block(int begin, int end, Sum& result)
{
    Sum local = 0;

    for (int i = begin; i < end; ++i)
    {
        local += i + 1;
    }

    result = local;
}
```

Za pokretanje koristimo ime funkcije i njene argumente:

```
    for (int t = 0; t < num_workers; ++t)
    {
        const int begin = Sum(n) * t / num_workers;
        const int end_index = Sum(n) * (t + 1) / num_workers;
        threads.emplace_back(sum_block, begin, end_index, ref(partial[t]));
    }

    for (auto& t : threads)
    {
        t.join();
    }

    const Sum parallel =
        accumulate(partial.begin(), partial.end(), Sum{0});
```

U izrazu za granice `Sum(n)` omogućava da se i množenje izračuna u širem tipu, pre deljenja i smeštanja granice u `int`. Poziv `ref(partial[t])` prosleđuje stvarni element vektora, tako da ga nit može izmeniti. Bez `ref`, mehanizam pokretanja niti bi pokušao da sačuva kopiju argumenta. Za ovu funkciju, koja traži referencu na rezultat, takav poziv se ne bi kompajlirao. Funkcija `ref` je u zaglavlju `<functional>`. Vektori `threads` i `partial` pripremljeni su pre ovog isečka. Svaka nit sabira lokalno i samo jednom upisuje `partial[t]`. Glavna nit čita te rezultate posle svih poziva `join`. Tada su niti završile i njihovi upisi su vidljivi glavnoj niti.

```
g++ -O3 -Wall -Wextra -pthread \
    src/threads/sum.cpp -o src/thread_sum
./src/thread_sum 3 10
./src/thread_sum 8 3
./src/thread_sum 4 0
```

Kada je podela jasna, uporediti sa `src/threads/sum_lambda.cpp`:

```
    for (int t = 0; t < num_workers; ++t)
    {
        const int begin = Sum(n) * t / num_workers;
        const int end_index = Sum(n) * (t + 1) / num_workers;
        threads.emplace_back([&, t, begin, end_index]
        {
            Sum local = 0;
            for (int i = begin; i < end_index; ++i)
            {
                local += i + 1;
            }
            partial[t] = local;
        });
    }
```

Granice i ID niti su kopirani, a vektor rezultata se deli. Isti testovi važe i za lambda varijantu:

```
g++ -O3 -Wall -Wextra -pthread \
    src/threads/sum_lambda.cpp -o src/thread_sum_lambda
./src/thread_sum_lambda 3 10
```

Rezultati treba da budu 55, 6 i 0. Ručno proveriti intervale i lokalne sume. Kada ima više niti nego iteracija, neki intervali su prazni. Ovaj primer služi za podelu posla. Za merenje koristiti pokretnu srednju vrednost, koja zaista čita ulazni niz; jednostavnu sumu kompajler može znatno da preobrazi.

2.4 Zadatak: pokretna srednja vrednost sa nitima

Dopuniti funkciju u fajlu `src/average/threads.cpp`:

```
void moving_average_threads(const Samples& x, int w,
    Samples& y, int num_workers);
```

Ulaz `x` i izlaz `y` već su pripremljeni. Podeliti *izlazne* indekse, jer jedna iteracija računa jedan izlaz. Broj izlaza je $M = N - W + 1$, pa u formuli za intervale koristiti M. Unutar petlje po izlaznom indeksu `i`, svaki prozor sabirati u lokalnu promenljivu:

```
int m = y.size();

double sum = 0;
for (int j = 0; j < w; ++j)
{
    sum += x[i + j];
}
y[i] = sum / w;
```

Koristiti `int` za indekse; program pre poziva proverava veličine. Granice bloka mogu se izračunati kao `m * t / num_workers` i `m * (t + 1) / num_workers`. Predviđene veličine i brojevi niti staju u ovaj proračun. Pokrenuti sve niti, a zatim ih sačekati u istoj funkciji.

Prvo izdvojiti proračun jednog bloka u običnu funkciju, na primer `average_block(x, w, y, begin, end)`. Kada je napišete, nit možete pokrenuti ovako:

```
threads.emplace_back(average_block, cref(x), w, ref(y), begin, end);
```

Poziv `cref(x)` prosleđuje ulaz po referenci samo za čitanje, a `ref(y)` omogućava upis u izlaz. Zatim isti poziv izraziti lambda funkcijom i proveriti da rezultati ostaju isti:

```
threads.emplace_back([&, begin, end]
{
    average_block(x, w, y, begin, end);
});
```

Kratka lambda funkcija ovde samo povezuje postojeću funkciju sa podacima. Prvo proveriti verziju sa običnom funkcijom; za merenja izabrati jednu verziju i zabeležiti koju koristite.

Za $N = 8$, $W = 3$ i $p = 4$ napisati četiri izlazna intervala. Nacrtati koje ulazne elemente čitaju dve susedne niti. Prozori mogu da se preklapaju: više niti sme da čita isti element. Svaki izlaz upisuje tačno jedna nit, pa upisi mogu teći paralelno. Svaka nit koristi svoju lokalnu sumu.

2.4.1 Provera i merenje

```
g++ -O3 -Wall -Wextra -pthread \
    src/average/thread_experiment.cpp src/average/sequential.cpp \
    src/average/threads.cpp -o src/thread_average
./src/thread_average --check
./src/thread_average 8 3 4 thread_8_3.csv
./src/thread_average 8 8 4 thread_8_8.csv
python3 scripts/run_experiments.py 2 --out results/lab2
python3 scripts/plot_results.py results/lab2
```

Poslednji argument zadaje fajl sa merenim vremenima. Ako se izostavi, koristi se `thread_average.csv`. Provere i sažetak prikazuju se na konzoli, a poruke o greškama idu kroz `cerr`. Provere obuhvataju poznate izlaze, negativne vrednosti, nejednake i prazne blokove. Dodati svoj mali test u funkciju `small_checks` u `src/average/thread_experiment.cpp`. Pokazati i intervale: tačan rezultat sam po sebi ne pokazuje da je svaki izlaz izračunat samo jednom.

Skripta koristi $N = 1000000$, $W = 64$ i 1, 2, 4 niti, do broja CPU-a dostupnih procesu. Po potrebi ograničiti broj opcijom `-max-workers 2`. Program u istom procesu zagreje svaku verziju,

zatim je meri pet puta, smenjujući redosled verzija. Svaki izlaz proverava van merenog intervala. Sekvencijalne funkcije su iste one koje ste napisali u prvoj vežbi.

Meri se ceo poziv funkcije. U verziji sa nitima to uključuje pripremu niti, njihovo kreiranje, računanje i čekanje. Priprema ulaznog i izlaznog niza, provere i ispis su van merenja. I posle zagrevanja ova verzija pri svakom pozivu ponovo stvara niti. I vreme sa jednom niti zato obuhvata njeno kreiranje i čekanje, pored samog proračuna.

Izračunati $S_p = T_{\text{direct}}/T_p$ i $E_p = S_p/p$, koristeći srednja vremena za isti ulaz. Tabela i grafik prikazuju ubrzanje prema direktnom postupku. Drugi grafik poredi vreme sekvencijalnog kliznog postupka sa paralelnim vremenom. Ako je taj odnos manji od 1, klizni postupak je brži. Izabrati novo W, na primer 8, predvideti promenu i ponoviti jedno merenje sa istim brojem niti. Sačuvati sva vremena i zabeležiti uslove rada računara.

2.5 Trka i zaključavanje

U fajlu `src/threads/race.cpp` sve niti menjaju isti brojač:

```
void increment(int n, long long& result)
{
    for (int i = 0; i < n; ++i)
    {
        ++result;
    }
}

threads.emplace_back(increment, n, ref(result));
```

Uvećanje običnog brojača čita staru vrednost i upisuje novu. Nacrtati redosled u kojem dve niti pročitaju istu staru vrednost i izgube jedno uvećanje. To objašnjava sukob; C++ program sa takvom trkom ima nedefinisano ponašanje. Za ispravnu verziju zaštititi ažuriranja brojača. Čekanje na kraju ne popravlja pristupe koji su se već sukobili.

Fajl `src/threads/mutex.cpp` štiti svako ažuriranje istim mutex-om:

```
void increment(int n, long long& result, mutex& counter_mutex)
{
    for (int i = 0; i < n; ++i)
    {
        lock_guard<mutex> guard(counter_mutex);
        ++result;
    }
}

threads.emplace_back(increment, n, ref(result), ref(counter_mutex));
```

Posle varijante sa običnom funkcijom uporediti lambda zapis istog zaštićenog ažuriranja u `src/threads/mutex_lambda.cpp`:

```
    for (int t = 0; t < num_workers; ++t)
    {
        threads.emplace_back([&]
        {
            for (int i = 0; i < n; ++i)
            {
                lock_guard<mutex> guard(counter_mutex);
                ++result;
            }
        });
    }
```

Nezaštićena lambda varijanta je u `src/threads/race_lambda.cpp`:

```
for (int t = 0; t < num_workers; ++t)
{
    threads.emplace_back([&]
    {
        for (int i = 0; i < n; ++i)
        {
            ++result;
        }
    });
}
```

U obe verzije zaštita zajedničkog brojača određuje ispravnost. Za lambda varijante u komandama za kompajliranje ispod zameniti `mutex.cpp` sa `mutex_lambda.cpp`, odnosno `race.cpp` sa `race_lambda.cpp`, i izabrati odgovarajuća imena izvršnih fajlova.

Objekat `lock_guard` zaključava pri nastanku i otključava na kraju svog opsega. Druga nit za to vreme čeka. Ispraviti trku na ovaj način, pa prepraviti brojač tako da niti broje lokalno, a glavna nit sabere rezultate posle čekanja. Za $p = 4$ i $n = 10000$ očekivani zbir je 40000. Objasniti zašto lokalna verzija izbegava zaključavanje u svakoj iteraciji. Ovo je vežba ispravnosti; merenja ubrzanja raditi na pokretnoj srednjoj vrednosti.

```
g++ -O3 -Wall -Wextra -pthread \
    src/threads/race.cpp -o src/thread_race
g++ -O3 -Wall -Wextra -pthread \
    src/threads/mutex.cpp -o src/thread_mutex
./src/thread_race 4 10000
./src/thread_mutex 4 10000
```

2.6 Pregled niti u GDB-u

Napraviti posebnu verziju za otklanjanje grešaka:

```
g++ -g -Og -pthread src/threads/hello.cpp \
    -o /tmp/thread_debug
gdb --args /tmp/thread_debug 4
```

Za običnu funkciju postaviti prekid po imenu, pa pregledati niti:

```
break worker
run
info threads
thread apply all backtrace
print id
continue
```

Izabrati postojeću nit komandom `thread ID`, gde je ID broj iz GDB spiska. GDB dodeljuje svoje ID vrednosti nitima koje prati. Pokazati stek izabrane niti i vrednost `id` koju joj je dodelio naš program. Isti pregled možete uraditi alatom u svom IDE-u: zaustaviti nit i pogledati njen ID i stek. Zaustavljanje menja tok izvršavanja, pa na ovoj verziji ne meriti brzinu.

2.6.1 Dodatno: otkrivanje trke alatom

Ako mašina podržava ThreadSanitizer, pokrenuti:

```
g++ -g -O1 -pthread -fsanitize=thread \
    src/threads/race.cpp -o /tmp/thread_race_tsan
/tmp/thread_race_tsan 4 10000
```

U poruci pronaći dva pristupa istom brojaču. Proveriti i verziju sa mutex-om, kompajliranu istim opcijama. Izostanak poruke nije dokaz da trke nema. Ako alat ne radi na mašini, analizirati pristupe u izvoru. Komande su detaljnije opisane u dokumentaciji GDB-a i GCC-a.

2.7 Pitanja o arhitekturi procesora

1. Zašto više niti ne mora da skrati vreme? Povezati odgovor sa količinom rada, kreiranjem niti i raspoloživim jezgrima.

2. Koje podatke niti samo čitaju, a koje menjaju? Objasniti zašto pokretna srednja vrednost ne zahteva zaključavanje izlaza.

3. Gde se u primeru brojača javlja čekanje? Mogu li susedni privatni rezultati pripadati istoj liniji keša iako nema trke?

4. Da li paralelni direktni postupak pobeđuje sekvencijalni klizni? Objasniti razliku između bolje podele rada i manjeg ukupnog rada.

Rezultat vežbe

- Pokazati sopstveni kod za podelu izlaza, intervale i provere, uključujući slučaj sa više niti nego izlaza.
- Objasniti trku, pokazati ispravku i pregled jedne niti u GDB-u ili odgovarajućem alatu svog IDE-a.
- Pokazati tabelu i grafik ubrzanja, granice merenja i proveru novog prozora.

2.8 Neobavezno: Pthreads

Standardna C++ biblioteka nudi tip `thread`. POSIX threads je drugi API za istu vrstu rada na Unix-like sistemima. Za poređenje su priloženi primeri u `src/pthreads/`. Primer pozdrava kompajlirati i pokrenuti ovako:

```
g++ -O3 -Wall -Wextra -pthread \
    src/pthreads/hello.cpp -o src/pthread_hello
./src/pthread_hello 4
```

Opcija kompajlera `-pthread` koristi se i za standardne C++ niti na Linux-u.